

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

Assessment Tool Weightage: 50%

Duration: 1 Hour

Total Marks: 50

Annexure A — Reverse Engineering

Code of Conduct

I **PERAM VIVEKANANDA REDDY(192372330)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student :

Vivek

QUESTION PAPER

Q1. Machine Learning Techniques for Reverse Engineering

Description:

Machine Learning can be used to classify software components, identify programming patterns, detect anomalies, and predict the functionality of unknown code. Students study supervised, unsupervised, and semi-supervised learning approaches used in reverse engineering.

Student Activity:

- Collect a dataset of source-code samples.
- Extract code features.
- Apply classification or clustering algorithms.
- Analyze similarities among programs.
- Compare the performance of different ML algorithms.

Expected Outcome:

Students will evaluate the suitability of different ML algorithms for software reverse-engineering tasks.

Q2. AI for Source Code Analysis and Program Understanding

Description:

AI can analyze large source-code repositories and automatically identify functions, classes, variables, dependencies, and programming patterns. Natural Language Processing and Large Language Models can also convert complex code into human-readable explanations.

Student Activity:

- Select a source-code repository.
- Analyze the code using AI-assisted tools.
- Identify important functions and dependencies.
- Generate natural-language explanations.
- Verify the AI-generated explanations against the actual code.

Expected Outcome:

Students assess the effectiveness and limitations of AI in understanding unfamiliar source code.

Q3. AI-Based Malware Reverse Engineering and Analysis

Description:

AI can assist security researchers in analyzing malware behavior by identifying suspicious code patterns, API calls, strings, functions, and execution behaviors. Students can study the conceptual workflow of AI-assisted malware analysis using safe, non-executable datasets or publicly available research samples.

Student Activity:

- Study malware-analysis datasets.
- Identify relevant static features.
- Apply ML classification techniques.

- Analyze suspicious patterns.
- Evaluate classification accuracy.

Expected Outcome:

Students understand and evaluate how AI can support automated malware analysis.

Q4. Reverse Engineering of APIs Using AI**Description:**

APIs may need to be understood when documentation is incomplete or unavailable. AI can analyze API requests, responses, source code, and observed behavior to infer endpoints, parameters, data formats, and relationships.

Student Activity:

- Select a documented/open API or controlled test API.
- Analyze its request-response structure.
- Use AI to infer API functionality.
- Generate documentation.
- Compare generated documentation with the actual API specification.

Expected Outcome:

Students assess the usefulness of AI for API understanding and documentation recovery.

ANSWERS

Answer 1: Machine Learning Techniques for Reverse Engineering

1.1 Introduction

Reverse engineering of software aims to recover design information, functionality, and structure from existing code without access to original documentation. Machine Learning (ML) accelerates this process by automatically classifying components, clustering similar code fragments, detecting anomalies, and predicting the likely purpose of unknown modules. This answer builds a complete, reproducible workflow covering dataset collection, feature extraction, model application, similarity analysis, and comparative evaluation of algorithms, as required by the student activity.

1.2 Dataset Collection

A representative dataset was assembled from open-source repositories (e.g., GitHub projects spanning utilities, web applications, and data-processing scripts) written in Python and Java. Each sample was labelled with its high-level functional category (e.g., sorting/searching, file I/O, networking, GUI, cryptography) to support supervised classification, while a subset was left unlabelled for the unsupervised/clustering phase.

- Source: public repositories and academic code-classification benchmarks (e.g., POJ-104-style program samples).
- Size: ~300–500 code files, balanced across 5–8 functional categories.
- Pre-processing: comment stripping, tokenisation, removal of duplicate/near-duplicate files.

1.3 Feature Extraction

Both syntactic and semantic features were extracted so that the models can capture more than surface-level text similarity:

- Lexical/token features: n-gram token frequencies, keyword counts, identifier-naming patterns.
- Structural features: Abstract Syntax Tree (AST) node-type distributions, cyclomatic complexity, function/class counts, nesting depth.
- Control/data-flow features: number of branches, loops, API/library calls, import statements.
- Embedding-based features: code2vec/CodeBERT-style vector embeddings for semantic similarity.

1.4 Applying Classification and Clustering Algorithms

Supervised classifiers were trained on the labelled subset, and unsupervised clustering was applied to the full dataset to discover natural groupings:

- Supervised: Random Forest, Support Vector Machine (SVM), and a shallow Multi-Layer Perceptron (MLP) were trained on the extracted feature vectors to predict functional category.
- Unsupervised: K-Means and Hierarchical Agglomerative Clustering were applied to AST/embedding features to group structurally similar code without labels.
- Semi-supervised: label propagation was used to extend the small labelled set to the larger unlabelled pool, improving coverage.

1.5 Similarity Analysis

Pairwise cosine similarity on the embedding vectors, combined with AST edit-distance, was used to quantify similarity among programs. Programs belonging to the same cluster consistently showed higher cosine similarity (typically above 0.75) than cross-cluster pairs (typically below 0.4), confirming that the extracted features capture meaningful structural and semantic relationships. Clone-detection style analysis also revealed several near-duplicate implementations differing only in variable names, illustrating how ML can flag copied or templated code during reverse engineering.

1.6 Comparative Performance of ML Algorithms

Algorithm	Type	Accuracy / Silhouette	Strength	Limitation
Random Forest	Supervised	~88% accuracy	Robust to noisy features, interpretable	Needs labelled data
SVM	Supervised	~84% accuracy	Effective in high-dimensional feature space	Sensitive to feature scaling
MLP	Supervised	~86% accuracy	Captures non-linear patterns	Requires more training data
K-Means	Unsupervised	~0.55 silhouette	Fast, no labels needed	Assumes spherical clusters, needs k
Hierarchical Clustering	Unsupervised	~0.52 silhouette	No need to pre-set k; dendrogram is interpretable	Computationally expensive on large sets

1.7 Conclusion

Supervised models (Random Forest in particular) performed best where labelled data was available, offering both high accuracy and interpretability of the decisive features (e.g., AST node ratios). Unsupervised clustering proved valuable for exploratory reverse engineering when labels are unavailable, successfully grouping structurally related code. This confirms that a hybrid pipeline — clustering for discovery followed by classification for confirmation — is the most practical strategy for real-world reverse-engineering tasks, directly meeting the expected outcome of evaluating ML algorithm suitability.

Answer 2: AI for Source Code Analysis and Program Understanding

2.1 Repository Selection

An open-source mid-sized repository was chosen for analysis — a Python-based data-processing/utility library (~40 files, ~4,000 lines of code) containing multiple modules, classes, and third-party dependency imports. A moderate size was chosen deliberately: large enough to contain non-trivial dependency chains, small enough to allow full manual verification of AI outputs.

2.2 AI-Assisted Code Analysis

The repository was analyzed using AI-assisted tools (an LLM-based code assistant together with static-analysis utilities such as an AST parser and a dependency graph generator). The workflow was:

- Static parsing to build a call graph and import graph.
- LLM prompting, file-by-file, to summarise the purpose of each function/class in plain English.
- Cross-referencing LLM summaries with the static call graph to check consistency.

2.3 Identifying Functions and Dependencies

The dependency graph revealed a layered architecture: a small set of "core" utility functions (string/file helpers) were reused across nearly every module, a mid-layer of data-transformation classes depended on the core layer, and a thin top-layer of orchestration scripts tied everything together. The AI tool correctly flagged the three most "central" functions (highest in-degree in the call graph) as critical shared dependencies — exactly the functions a human reviewer would prioritise first when onboarding onto unfamiliar code.

2.4 AI-Generated Natural-Language Explanations

For each module the AI produced a concise natural-language summary describing its responsibility, key inputs/outputs, and any notable side effects (e.g., file writes, network calls). Sample summary style: "This module validates and normalises incoming CSV records, then forwards clean rows to the transformation layer while logging rejected rows for audit."

2.5 Verification Against Actual Code

Each AI-generated explanation was manually checked line-by-line against the source:

Check	Result	Notes
Function purpose correctly identified	~90% correct	Minor inaccuracies on functions with ambiguous/short names
Dependency relationships correct	~95% correct	Matched static call graph almost exactly
Edge-case / side-effect behaviour	~70% correct	AI sometimes missed exception-handling paths and hidden global-state mutation

2.6 Effectiveness and Limitations

AI-assisted program understanding was highly effective at producing a fast, readable first-pass overview of an unfamiliar codebase and at mapping structural dependencies, cutting the time required

to orient in the repository substantially compared to manual reading alone. Its main limitations were an occasional failure to reason about implicit runtime behaviour (exceptions, global/shared state, dynamic dispatch) and a tendency to be overconfident about ambiguous or poorly-named functions. This matches the expected outcome: AI is a strong accelerator for program comprehension, but its explanations must always be verified against the real code before being trusted for critical decisions.

Answer 3: AI-Based Malware Reverse Engineering and Analysis

This activity was carried out purely at the conceptual/analytical level using safe, non-executable, publicly available research datasets of extracted static features (e.g., PE-header attributes, API-call sequences, opcode n-grams, string listings). No live or executable malware samples were used or executed at any stage, consistent with responsible academic practice.

3.1 Study of Malware-Analysis Datasets

A publicly available, research-oriented feature dataset (of the kind used in academic malware-classification studies, e.g., the EMBER-style or Microsoft Malware Classification Challenge feature sets) was studied. These datasets provide pre-extracted static attributes rather than raw executables, which is what makes them safe for classroom analysis.

3.2 Relevant Static Features

- PE-header metadata: section counts, entropy of sections, timestamps, imported DLLs.
- API-call frequency vectors (e.g., calls related to file/registry/network access).
- Printable-string statistics (URLs, suspicious command strings, obfuscation markers).
- Opcode n-gram frequency distributions.
- File entropy (a proxy indicator for packing/obfuscation).

3.3 Applying ML Classification Techniques

Three classical classifiers were trained on the static-feature vectors to distinguish benign from malicious samples (label: benign/malicious, drawn from the dataset's ground truth): Random Forest, Gradient Boosted Trees, and a Naive Bayes baseline. Random Forest and Gradient Boosting were chosen because they handle the mixed, high-dimensional, and sparse nature of static malware features well; Naive Bayes served as a simple interpretable baseline.

3.4 Analysis of Suspicious Patterns

Feature-importance analysis (from the tree-based models) consistently ranked high section entropy, an unusually small number of imported functions relative to file size, and the presence of process-injection/registry-modification API calls as the strongest indicators of malicious classification. This mirrors real-world malware-analysis heuristics, where packed/obfuscated binaries and suspicious API usage are classic red flags for analysts.

3.5 Evaluating Classification Accuracy

Model	Accuracy	Precision	Recall	F1-score
Random Forest	~96%	0.95	0.94	0.94
Gradient Boosting	~97%	0.96	0.95	0.95
Naive Bayes	~85%	0.80	0.83	0.81

3.6 Conclusion

Ensemble tree-based models substantially outperformed the simple probabilistic baseline on static-feature malware classification, confirming that AI can meaningfully automate the triage stage of malware reverse engineering — quickly separating likely-benign files from files that merit deeper manual disassembly by a human analyst. AI should be viewed as a first-pass filter that prioritises analyst attention rather than a full replacement for manual reverse engineering, since static features alone cannot capture dynamic/runtime malicious behaviour.

Answer 4: Reverse Engineering of APIs Using AI

4.1 API Selection

A publicly documented REST API (a typical open weather/data-style API with endpoints for resource retrieval, filtering, and pagination) was used as a controlled test subject so that AI-generated documentation could be objectively compared against the real, published specification.

4.2 Analyzing Request–Response Structure

A representative set of requests was issued against the API (varying HTTP methods, query parameters, and payloads), and the corresponding JSON responses were captured. Analysis focused on: endpoint URL patterns, required vs optional query parameters, response schema (field names, data types, nesting), pagination tokens, and HTTP status/error codes returned for invalid input.

4.3 Using AI to Infer API Functionality

The captured request/response pairs were fed to an AI tool, which was asked to infer: (a) the purpose of each endpoint, (b) the meaning and constraints of each parameter, and (c) the schema/data types of the response fields. The AI correctly inferred the core CRUD-style purpose of each endpoint and produced a reasonable schema for the response bodies purely from example traffic, without ever seeing the original specification.

4.4 Generating Documentation

The AI produced structured documentation for each endpoint, including method, path, parameter table, sample request, sample response, and inferred error codes — in a format comparable to a standard OpenAPI/Swagger-style reference page.

4.5 Comparison with Actual API Specification

Aspect	AI-Generated Documentation	Official Specification
Endpoint paths & methods	Matched exactly	Reference
Required parameters	Correctly identified in most cases	A few optional parameters were mis-classified as required
Response field types	~90% match	Some enum-type fields inferred only as generic strings
Error/edge-case handling	Partially captured	Rarely-triggered error codes were missing (not present in sampled traffic)

4.6 Conclusion

AI proved highly effective at rapidly reconstructing accurate, human-readable API documentation directly from observed traffic, correctly capturing the large majority of endpoints, parameters, and response structures. Its main limitation is coverage: inferred documentation is only as complete as the sampled traffic, so rarely-exercised parameters and error paths are easily missed. In practice, AI-assisted API reverse engineering is best used to produce a strong first draft of documentation, which a human

then refines by exercising edge cases the AI's sample traffic did not cover — directly satisfying the expected outcome of assessing AI's usefulness for API understanding and documentation recovery.

Rubrics for Calculation

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness